

SUPPLEMENTARY · MODULE 5

Integration and Event-Driven Messaging

Connect services without a distributed monolith: Kafka and Spring events, idempotent consumers, outbox, and the failure modes that show up in a payment flow.



COMMUNICATION CHALLENGES IN DISTRIBUTED SYSTEMS

Latency, service unavailability, duplicate messages, and versioning turn simple service-to-service calls into real engineering problems.



COMMON INTEGRATION CHALLENGES

- Network latency, service unavailability, and timeout handling
- Duplicate messages, data consistency, and API versioning
- Contract changes, error handling, and security between services



BANKING EXAMPLE

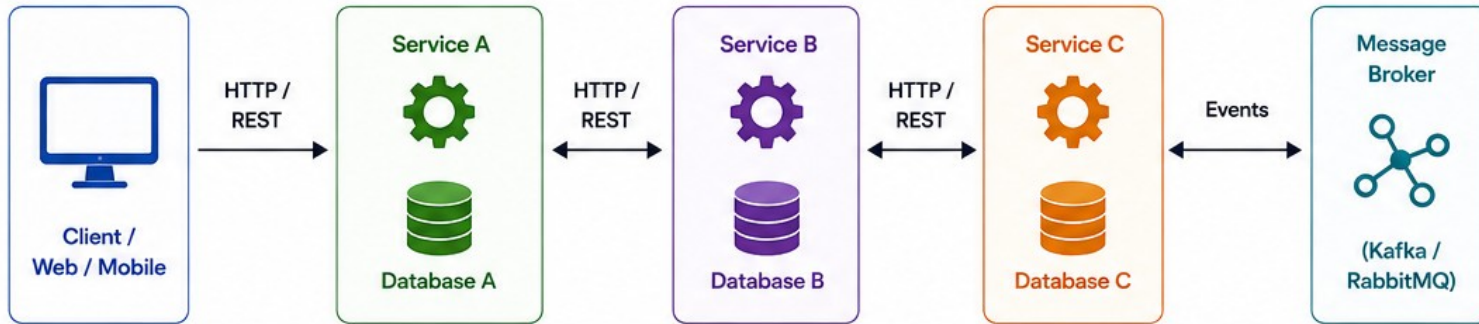
- If Account Service is unavailable, Transaction Service must choose: reject, retry, queue as pending, use cached data, or publish an event for later
- The correct answer depends on the business requirement, not the technology

KEY TAKEAWAY Latency, service unavailability, duplicate messages, and versioning turn simple service-to-service calls into real engineering problems.

Integration Challenges

Building distributed systems means integrating independent services—introducing complexity beyond a monolith.

Typical Microservices Integration Landscape



Why Integration Becomes Harder



Multiple Services

More services = more dependencies and integration points.



Network Calls

Unreliable networks introduce latency, timeouts, and failures.



Independent Data

Data is distributed; no single source of truth.



Asynchronous Nature

Events and eventual consistency make coordination difficult.



Partial Failures

Some services succeed while others fail—hard to handle consistently.

Key Integration Challenges



1. Network Latency and Timeouts

- Network calls are slower than in-process calls.
- Timeouts can cause failures even when services are healthy.



2. Service Unreliability

- Services can crash, restart, or become unavailable.
- Integration must handle retries and failover.



3. Data Consistency Issues

- No shared database.
- Maintaining consistency across services is hard.
- Leads to eventual consistency.



4. Transaction Management

- Distributed transactions are complex and costly.
- Requires patterns like Saga and Outbox.



5. Versioning and Compatibility

- Services evolve independently.
- Backward/forward compatibility must be maintained.



6. Security and Governance

- Secure communication between services.
- Authentication, authorization, and auditing are essential.

How We Address These Challenges



Resilience Patterns

Retries, timeouts, circuit breakers, bulkheads.



Asynchronous Communication

Use events and messaging to decouple services.



Data Ownership

Each service owns its data; design for eventual consistency.



Saga Pattern

Manage distributed transactions with compensating actions.



API Contracts & Versioning

Define clear contracts and version APIs.



Observability

Centralized logging, metrics, tracing, and alerting.

SERVICE-TO-SERVICE COMMUNICATION OPTIONS · SYNCHRONOUS COMMUNICATION WITH HTTP



TWO BROAD INTEGRATION STYLES

- **Synchronous communication:** commonly implemented with REST APIs, caller waits for a response
- **Asynchronous communication:** commonly implemented with messaging, events, and brokers
- Most real banking workflows use a deliberate mix of both



REST API CALL FLOW

- Client -> Transaction Service -> Account Service -> Account Database -> response flows back
- Easy to understand, but creates a runtime dependency between the two services



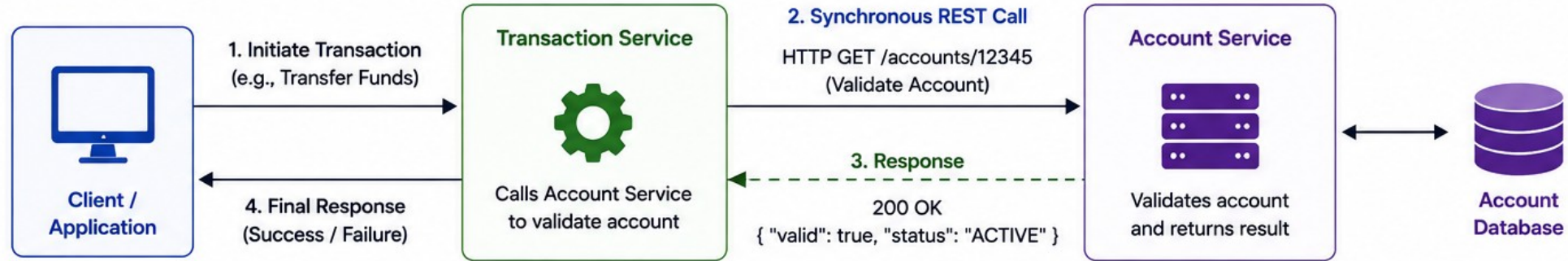
BEST PRACTICES

- Set connection and read timeouts on every synchronous call
- Avoid long chains of synchronous calls; retry only safe (idempotent) operations

KEY TAKEAWAY Every integration decision starts with one question: does the caller need an answer right now, or can the work happen later? • Transaction Service calling GET /accounts/5001 is simple and testable, but Account Service must be healthy for the call to succeed.

Synchronous Communication (REST Account Validation)

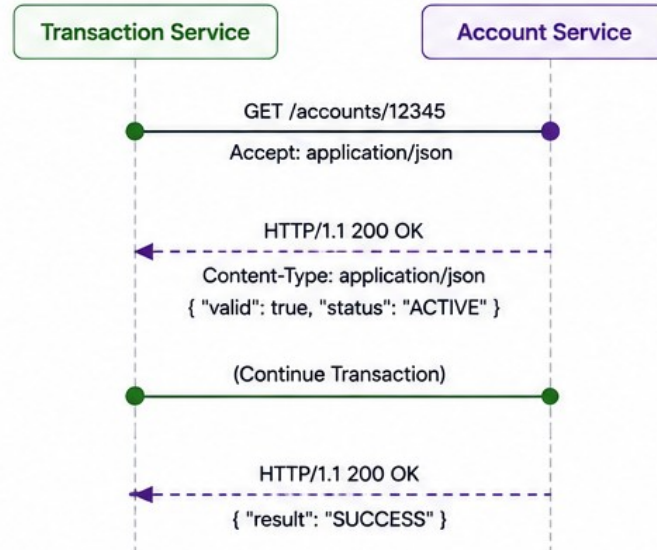
A service makes a blocking REST call to another service and waits for the response.



Synchronous = Blocking

The Transaction Service waits for the Account Service response.
If the Account Service is slow or unavailable, the transaction is delayed or fails.

HTTP Sequence (Example)



Key Characteristics



Blocking

Caller waits for the response before continuing.



Real-time

Immediate validation and up-to-date data.



Simple to Implement

Uses standard HTTP/REST APIs.



Tight Coupling

Caller depends on the availability of the callee.



Latency Sensitive

Performance depends on network and service response time.

Example REST Call

Request

```
GET /accounts/12345 HTTP/1.1
Host: account-service
Accept: application/json
Authorization: Bearer eyJhbGciOi...
```

Response (200 OK)

```
{
  "accountId": "12345",
  "valid": true,
  "status": "ACTIVE",
  "accountType": "CHECKING",
  "balance": 1250.75
}
```

Advantages

- ✓ Simple and easy to understand
- ✓ Immediate consistency
- ✓ Uses standard REST and HTTP
- ✓ Good for real-time validations

Disadvantages

- ✗ Blocking and increases latency
- ✗ Cascading failures if callee is down
- ✗ Lower resilience and scalability
- ✗ Higher network dependency

DECLARATIVE HTTP CLIENTS · SPRING RESTCLIENT AND WEBCLIENT · BLOCKING VS. NON-BLOCKING COMMUNICATION



WHY DECLARATIVE CLIENTS HELP

- Define the remote service contract as a Java interface with annotations
- The framework generates the HTTP call implementation automatically
- Reduces boilerplate compared to manually building requests for every call



RESTCLIENT: THE MODERN SYNCHRONOUS CLIENT

- Blocking, imperative API built on Spring's `HttpService` and `HttpClient`
- Ideal for traditional MVC applications and simpler use cases



WEBCLIENT: THE REACTIVE, NON-BLOCKING CLIENT

- Non-blocking, reactive API built on Project Reactor (Mono/Flux) and Netty
- Ideal for reactive applications, high concurrency, and streaming APIs (SSE)



BLOCKING COMMUNICATION

- The calling thread waits until the response is received
- Simple, imperative model; consumes one thread per request during the wait
- Good fit for traditional MVC apps and low-to-moderate traffic



NON-BLOCKING COMMUNICATION

- The calling thread is not blocked and can do other work while waiting
- Uses fewer threads, enabling much higher concurrency for I/O-bound workloads

KEY TAKEAWAY Annotate an interface with the target endpoints and the client framework generates the HTTP call implementation for you. • `RestClient.builder().get().uri(...).retrieve().body(...)` and `WebClient's Mono`-returning equivalent share a similar fluent API for different threading models. • A blocking call holds one thread per request until the response arrives; a non-blocking call frees the thread to do other work while it waits.

REQUEST TIMEOUTS AND CONNECTION MANAGEMENT · SERVICE DISCOVERY AND CLIENT-SIDE LOAD BALANCING



TIMEOUTS TO CONFIGURE

- **Connection timeout:** max time to establish a connection to the server
- **Connection request timeout:** max time to wait for a connection from the pool
- **Read/response timeout:** max time to wait for data after the request is sent



CONNECTION POOLING ESSENTIALS

- Reuse connections instead of creating new ones -- reduces latency and resource usage
- Tune pool size based on concurrency and downstream capacity; evict idle connections



HOW IT WORKS

1. Client queries the service registry for available instances of a service
2. Registry returns a healthy instance list with metadata
3. Client's load balancer selects an instance (round robin, random, least connections, weighted)
4. Client sends the request to the selected instance and retries elsewhere if it's unhealthy

KEY TAKEAWAY Connection, connection-request, read, and write timeouts each guard a different phase of an HTTP call from waiting forever. • A service registry (Eureka, Consul, etcd) tracks live instances; the client's load balancer then picks a healthy one for each request.

API GATEWAY RESPONSIBILITIES · AVOIDING CHATTY SERVICE COMMUNICATION · HANDLING PARTIAL FAILURES



CORE RESPONSIBILITIES

- Request routing, authentication and authorization, request/response filtering
- Rate limiting and throttling, caching, security enforcement (TLS, WAF)
- Monitoring and logging, resilience (timeouts, retries, circuit breakers), API versioning



BEST PRACTICES

- Keep the gateway stateless, highly available, and free of business logic
- Monitor performance and set SLOs for the gateway itself



WHAT MAKES COMMUNICATION CHATTY

- Many small requests between services to complete one business operation
- High round-trip count increases latency, resource usage, and failure points



STRATEGIES TO REDUCE CHATTINESS

- Design coarse-grained APIs that return all data a client needs for a use case
- Use aggregation services, API composition, and caching to cut round-trips



STRATEGIES TO HANDLE PARTIAL FAILURES

- Timeouts to fail fast; retries with exponential backoff and jitter
- Circuit breakers to stop calling a failing service temporarily
- Fallbacks, bulkhead isolation, rate limiting, and graceful degradation

KEY TAKEAWAY Ten core responsibilities, from request routing to API versioning, all sit at the edge instead of being duplicated inside every microservice. • Coarse-grained APIs, aggregation services, and API composition each reduce round-trips that would otherwise multiply latency and failure points. • When Inventory Service is down, the rest of the system should degrade gracefully -- show cached data, hide a section -- rather than crash entirely.

DATA SERIALIZATION AND SCHEMA EVOLUTION · SELECTING THE RIGHT COMMUNICATION PATTERN



COMMON SERIALIZATION FORMATS

- **JSON:** human-readable, widely used for REST APIs and general communication
- **Avro:** compact, schema-based, strong typing -- common for Kafka event streaming
- **Protocol Buffers:** very compact and high performance for low-latency systems



SCHEMA EVOLUTION STRATEGIES

- Additive changes and optional fields are backward and forward compatible
- Deprecate before removing a field; version the schema for major changes



DECISION GUIDE

- Immediate response required? Yes -> synchronous. No -> asynchronous
- Low coupling important? -> asynchronous. Strong coupling acceptable? -> synchronous
- Strong/immediate consistency needed? -> synchronous. Eventual is fine? -> asynchronous

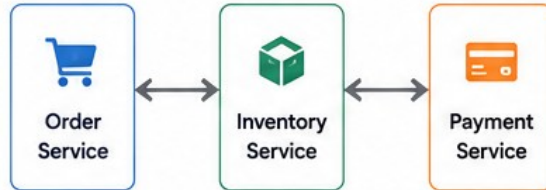
KEY TAKEAWAY Additive, optional changes stay backward and forward compatible; removing or renaming a field breaks consumers on both sides. • Immediate response needed, low coupling tolerance, or critical-path failure sensitivity each point toward synchronous or asynchronous differently.

Why Use Messaging Between Microservices?

Messaging enables services to communicate asynchronously, improving decoupling, scalability, resilience, and responsiveness of distributed systems.

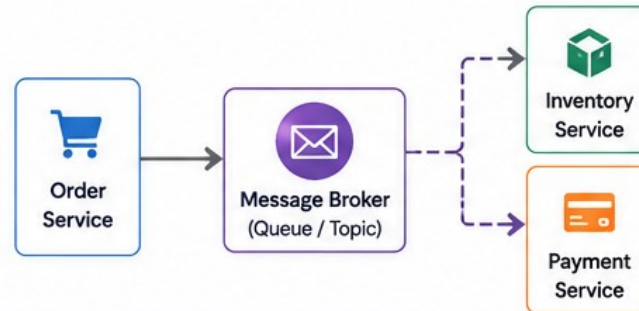


Without Messaging (Synchronous Calls)



- ✗ Tight coupling between services
- ✗ A service must be available for the call to succeed
- ✗ Failure in one service can cascade to others
- ✗ Limited scalability under high load
- ✗ User request is blocked waiting for responses

With Messaging (Asynchronous)



- ✓ Loose coupling between services
- ✓ Services communicate without waiting
- ✓ Failures are isolated; improves resilience
- ✓ Higher scalability and throughput
- ✓ Better user experience and system responsiveness
- ✓ Enables event-driven and real-time architectures

Key Benefits of Using Messaging

- Loose Coupling**
Services are independent and can evolve without impacting each other.
- Scalability**
Handle more load by scaling consumers independently of producers.
- Resilience**
Buffers absorb spikes and protect services from temporary failures.
- Improved Responsiveness**
Producers don't wait—users get faster acknowledgments.
- Better Throughput**
Background processing increases overall system throughput.
- Event-Driven Architecture**
Enables real-time reactions and integration across domains and systems.

Common Use Cases

- Order Processing**
Process orders without blocking inventory or payment.
- Notifications**
Send emails/SMS without delaying user requests.
- Event Streaming**
Publish domain events for analytics and monitoring.
- System Integration**
Decouple systems across teams or technologies.
- Background Tasks**
Run long-running or scheduled tasks asynchronously.



Key Takeaway

Messaging enables microservices to communicate in a scalable, resilient, and flexible way by decoupling services and allowing asynchronous, event-driven interactions.



When Not to Use Messaging

For simple, low-latency, request/response interactions where immediate consistency is required.

INTRODUCTION TO ASYNCHRONOUS MESSAGING

A producer sends a message to a broker and moves on; the broker stores and delivers it independently, whenever the consumer is ready.



HOW ASYNCHRONOUS MESSAGING WORKS

1. Producer sends a message to a broker (queue or topic)
2. Broker stores and delivers the message independently
3. Consumer receives the message when it is ready to process it

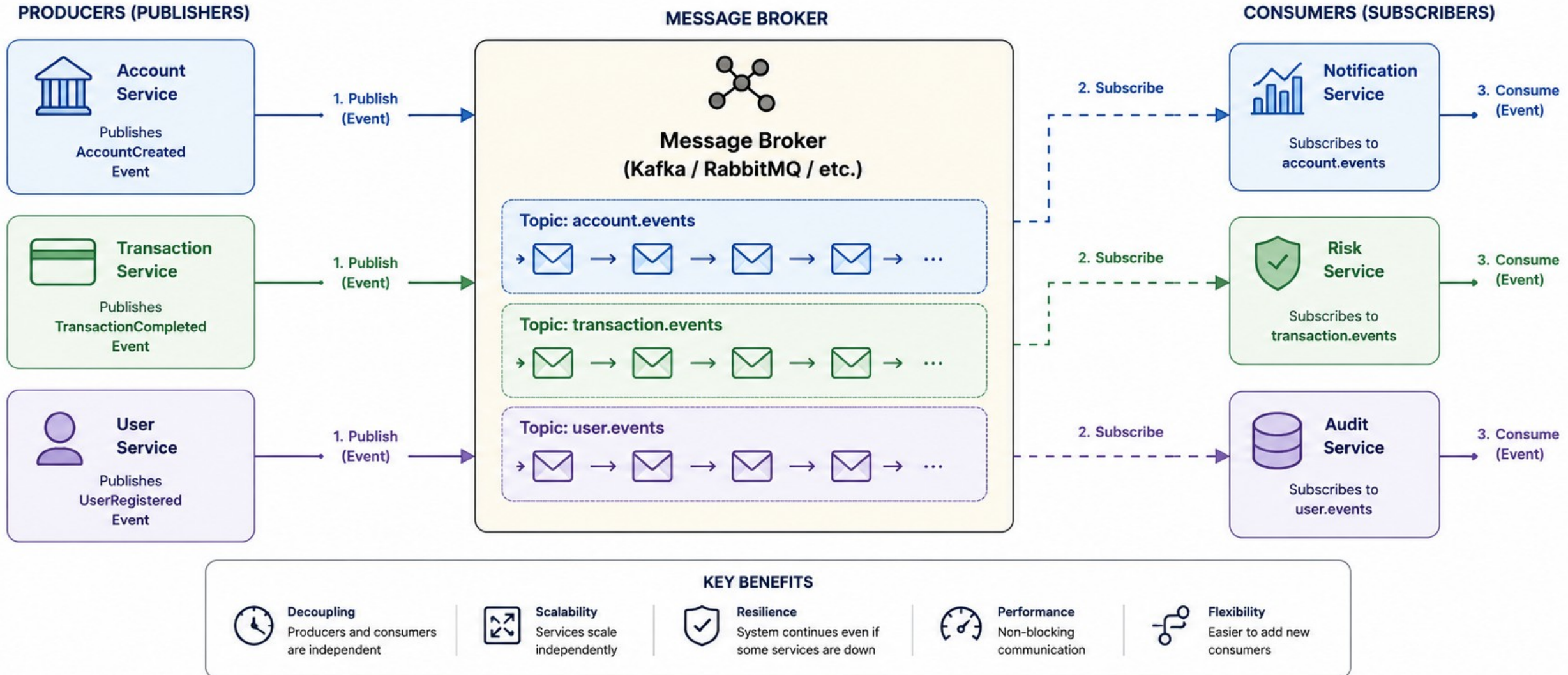


KEY CHARACTERISTICS

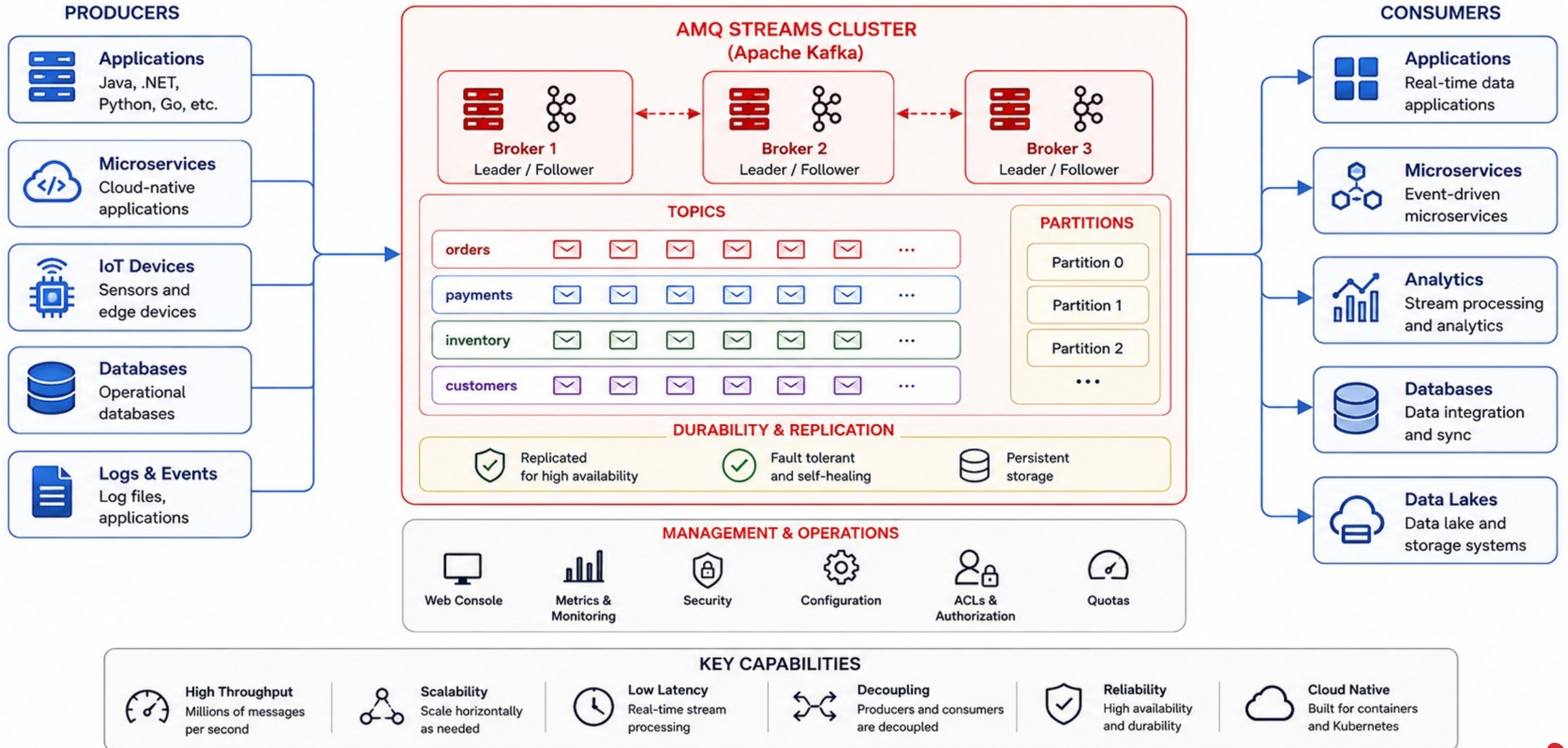
- **Decoupling in time:** sender and receiver don't need to be available simultaneously
- Improved scalability, resilience during outages, and eventual processing

KEY TAKEAWAY A producer sends a message to a broker and moves on; the broker stores and delivers it independently, whenever the consumer is ready.

ASYNCHRONOUS COMMUNICATION AND EVENT-DRIVEN ARCHITECTURE



Red Hat AMQ Streams Overview



MESSAGE BROKERS AND THEIR RESPONSIBILITIES · QUEUES, TOPICS, AND SUBSCRIPTIONS



CORE RESPONSIBILITIES OF A BROKER

- Message ingestion, durable storage, routing and delivery to the correct consumers
- Delivery guarantees (at-most-once, at-least-once, exactly-once)
- Acknowledgment management, buffering/flow control, monitoring and management



QUEUES (POINT-TO-POINT)

- Each message is delivered to and consumed by one consumer
- Ideal for load balancing and task distribution (order processing, background jobs)

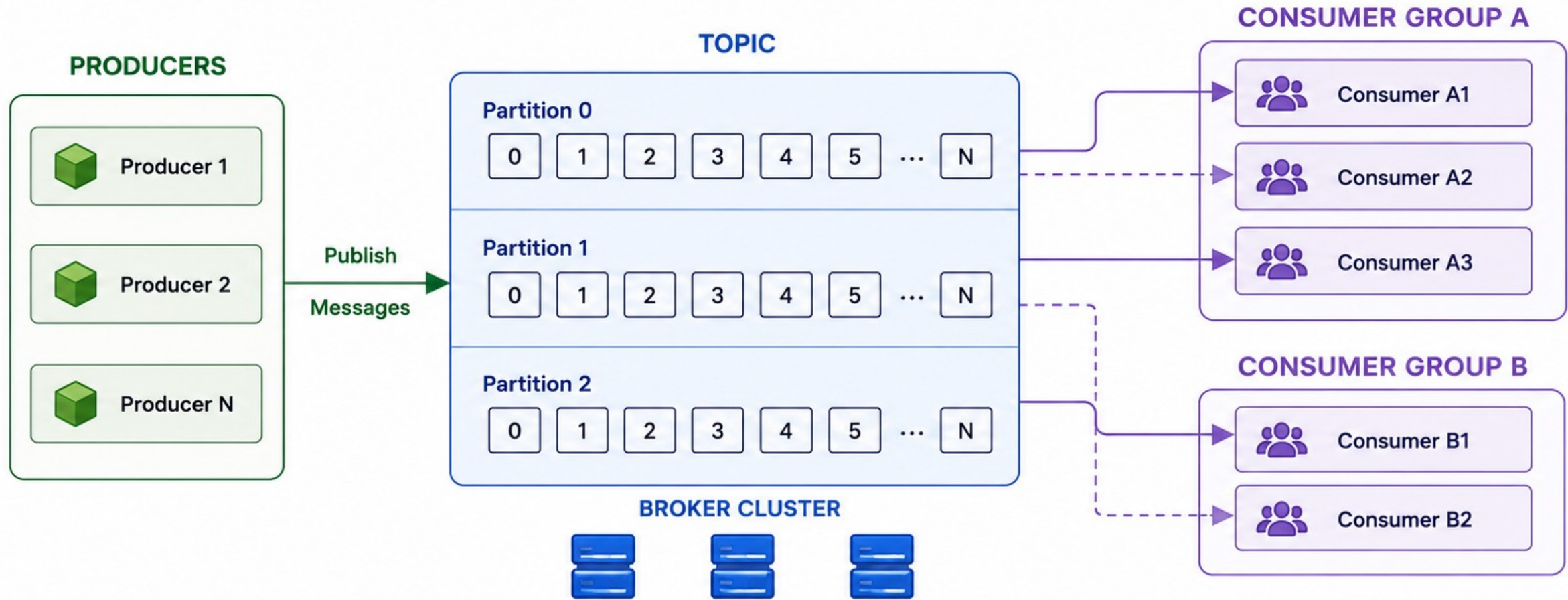


TOPICS (PUBLISH/SUBSCRIBE)

- Each message is delivered to all interested subscribers
- Ideal for event broadcasting and notifications; subscribers join or leave independently

KEY TAKEAWAY Without a broker, Account Service would need to know and call every consumer directly -- with one, it just publishes and moves on. • A queue removes a message once one consumer processes it; a topic keeps delivering the same message to every interested subscriber.

Producers, Consumers, and Consumer Groups



PRODUCERS

Producers publish messages to a topic.



TOPIC & PARTITIONS

Topics are split into partitions. Messages in a partition are ordered by offset.



CONSUMERS & CONSUMER GROUPS

Consumers read messages from partitions. Within a consumer group, each partition is consumed by only one consumer.

POINT-TO-POINT MESSAGING · PUBLISH-AND-SUBSCRIBE MESSAGING · COMMANDS, EVENTS, AND NOTIFICATIONS

HOW IT WORKS

1. Producer sends a message to a queue
2. Broker stores the message in the queue
3. A consumer retrieves and processes the message
4. The message is acknowledged and removed; unacknowledged messages can be redelivered

WHEN TO USE

- When multiple services need the same event or update
- When you want to decouple producers from consumers entirely

HOW IT WORKS

1. A publisher sends a message to a topic
2. The broker stores the message in the topic
3. All active subscribers receive a copy and process it independently

THREE MESSAGE TYPES

- **Commands:** request an action, sent to a single responsible service (point-to-point)
- **Events:** describe a fact that already happened, published to a topic, should be immutable
- **Notifications:** inform or alert, no action required, often consumed by external systems or UIs

KEY TAKEAWAY A message sits in the queue until acknowledged; if the consumer fails before acknowledging, the message becomes available again for another worker. • New subscribers can start receiving future messages without any change to the publisher -- the two sides stay completely decoupled. • A CreateOrderCommand asks for an action, an OrderCreatedEvent records that it happened, and an OrderShippedNotification simply informs interested parties.

EVENT-DRIVEN ARCHITECTURE · EVENT NOTIFICATION VS. EVENT-CARRIED STATE TRANSFER



EDA DESIGN PRINCIPLES

- **Events over commands:** share facts, not requests
- **Immutability:** events should never be changed or deleted
- Asynchronous first, idempotent consumers, single responsibility per event



EVENT NOTIFICATION (EN)

- A lightweight message that announces a fact -- contains only identifiers and metadata
- Subscriber must call back to the publishing service to get the data it needs



CHALLENGES AND CONSIDERATIONS

- Eventual consistency, added complexity, and the need for end-to-end monitoring
- Message ordering and duplicate delivery both require deliberate design

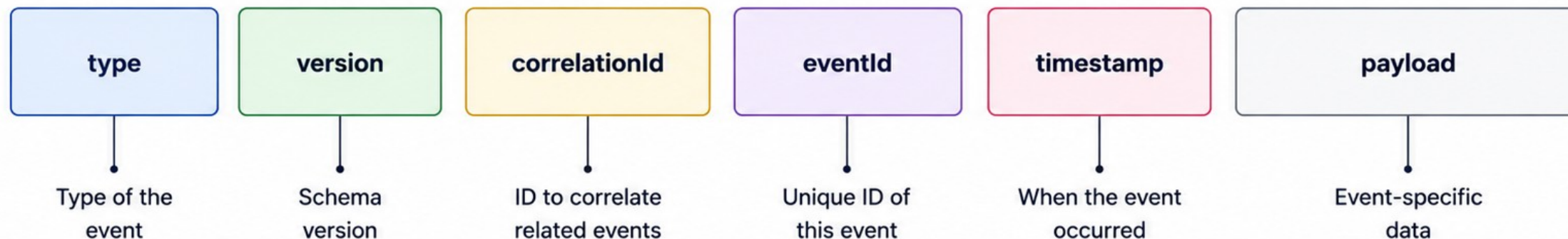


EVENT-CARRIED STATE TRANSFER (ECST)

- A self-contained message that includes the data subscribers need to process it
- Lower coupling and better resilience, at the cost of larger messages and data duplication

KEY TAKEAWAY Produce, publish, subscribe, react -- this four-step cycle is what lets services evolve, scale, and deploy independently of each other. • Event Notification keeps payloads small but adds a lookup call; Event-Carried State Transfer is self-contained but larger and harder to evolve.

Event Schema



Field	Description	Example	Notes
type	Type of the event	account.created	Event name in dot notation
version	Schema version of the event	1.0	Increment when schema changes in a non-backward-compatible way
correlationId	ID to correlate related events across services	550e8400-e29b-41d4-a716-446655440000	Same ID used by all events in the same business transaction/flow
eventId	Unique identifier of this event	c1f2d3e4-5678-4abc-9def-0123456789ab	Must be globally unique (UUID recommended)
timestamp	Timestamp when the event occurred (UTC)	2025-05-09T14:23:45.123Z	ISO 8601 format in UTC

Message Schema Design

A **message schema** defines the structure, data types, and validation rules for **messages** exchanged between producers and consumers. Well-designed schemas enable compatibility, evolution, and reliable processing.

EXAMPLE: ORDER_CREATED EVENT

JSON Schema (Example)

```
{
  "type": "object",
  "required": ["eventId", "eventType", "occurredAt", "order"],
  "properties": {
    "eventId": { "type": "string", "format": "uuid" },
    "eventType": { "type": "string", "enum": ["ORDER_CREATED"] },
    "occurredAt": { "type": "string", "format": "date-time" },
    "version": { "type": "integer", "minimum": 1 },
    "source": { "type": "string" },
    "order": {
      "orderId": { "type": "string" },
      "customerId": { "type": "string" },
      "orderDate": { "type": "string", "format": "date" },
      "totalAmount": { "type": "number" },
      "currency": { "type": "string", "pattern": "^[A-Z]{3}$" },
      "items": {
        "type": "array",
        "items": {
          "type": "object",
          "required": ["productId", "quantity", "unitPrice"],
          "properties": {
            "productId": { "type": "string" },
            "quantity": { "type": "integer", "minimum": 1 },
            "unitPrice": { "type": "number" }
          }
        }
      }
    }
  }
}
```

SCHEMA STRUCTURE

-  **Metadata**
Information about the event and its origin.
-  **Payload**
Business data relevant to the event.
-  **Identifiers**
Unique IDs for tracing, deduplication, and correlation.
-  **Version**
Schema version to manage compatibility.
-  **Extensibility**
Allow optional fields for future needs.

SCHEMA DESIGN CHECKLIST

Area	Key Considerations
 Metpdse & Naming	- Use descriptive event/message names. - Follow a consistent naming convention.
 Fields & Types	- Choose appropriate data types. - Avoid ambiguous types (e.g., string for numbers).
 Required vs. Optional	- Mark only truly essential fields as required. - Optional fields must have safe defaults.
 Validation	- Define constraints (min/max, pattern, enum). - Validate at the producer and consumer.
 Compatibility	- Backward compatibility: consumers can read new messages. - Forward compatibility: old messages can be read by new consumers.
 Documentation	- Document fields, types, and meanings. - Maintain schema in a registry.
 Governance	- Review changes and versions. - Deprecate fields carefully.

DESIGN PRINCIPLES

-  **Clarity**
Names are clear, purpose is well-defined.
-  **Consistency**
Use standard conventions and data types.
-  **Extensibility**
Design for future fields and changes.
-  **Validation**
Validate early to prevent bad messages.
-  **Compatibility**
Ensure backward and forward compatibility.

BENEFITS OF GOOD SCHEMA DESIGN

-  **Faster integration and onboarding**
-  **Fewer runtime errors and message rejects**
-  **Safe schema evolution over time**
-  **Better governance and observability**
-  **Stronger reliability and trust**



Remember:
Your schema is a contract. Design it with your consumers in mind, not just your current needs.

Published Event Vocabulary

Category	Event Name	Description	Key Attributes (Examples)	Published By	Consumed By
 Account Events	account.created	An account is created	accountId, customerId, accountType, currency, createdAt, correlationId	Account Service	Customer Service, Notification Service, Risk Service
	account.updated	An account is updated	accountId, changedBy, changes, updatedAt, correlationId	Account Service	Notification Service, Audit Service
	account.closed	An account is closed	accountId, closedBy, reason, closedAt, correlationId	Account Service	Customer Service, Notification Service, Audit Service, Risk Service
 Transaction Events	transaction.created	A transaction is created	transactionId, accountId, amount, currency, type, createdAt, correlationId	Transaction Service	Risk Service, Notification Service
	transaction.approved	A transaction is approved	transactionId, approvedBy, approvedAt, correlationId	Transaction Service	Notification Service, Audit Service
	transaction.settled	A transaction is settled	transactionId, settledAt, settlementRef, correlationId	Transaction Service	Account Service, Notification Service, Audit Service
	transaction.failed	A transaction fails	transactionId, reason, failedAt, correlationId	Transaction Service	Risk Service, Notification Service, Audit Service
 Customer Events	customer.registered	A customer is registered	customerId, name, email, phone, registeredAt, correlationId	Customer Service	Account Service, Notification Service, Risk Service
	customer.updated	A customer is updated	customerId, changedBy, changes, updatedAt, correlationId	Customer Service	Notification Service, Audit Service
	customer.deactivated	A customer is deactivated	customerId, deactivatedBy, reason, deactivatedAt, correlationId	Customer Service	Account Service, Notification Service, Risk Service
 Risk Events	risk.evaluated	A risk evaluation is completed	evaluationId, entityType, entityId, score, evaluatedAt, correlationId	Risk Service	Notification Service, Audit Service
	risk.flagged	A risk is flagged	flagId, entityType, entityId, reason, severity, flaggedAt, correlationId	Risk Service	Notification Service, Case Management Service
	risk.cleared	A risk flag is cleared	flagId, clearedBy, clearedAt, correlationId	Risk Service	Notification Service, Audit Service
 Notification Events	notification.sent	A notification is sent	notificationId, channel, to, subject, sentAt, correlationId	Notification Service	Audit Service
	notification.failed	A notification fails	notificationId, channel, to, reason, failedAt, correlationId	Notification Service	Audit Service

MESSAGE PRODUCERS AND CONSUMERS · DELIVERY GUARANTEES

THE THREE GUARANTEES

GUARANTEE	DUPLICATES?	MESSAGE LOSS?	USE WHEN
At-Most-Once	Never	Possible	Occasional loss OK, duplicates not (metrics, logging)
At-Least-Once	Possible	No (normal conditions)	Every message must be delivered (orders, payments)
Exactly-Once	Never	Never (normal conditions)	Both loss and duplicates unacceptable (financial transactions)

PRODUCER KEY RESPONSIBILITIES

- Create messages with correct data and schema; choose the destination (topic or queue)
- Set headers, metadata, and partitioning keys; handle errors and retries when publishing

CONSUMER KEY RESPONSIBILITIES

- Subscribe to the right topic or queue; process messages correctly and efficiently
- Acknowledge or commit processing; handle failures and ensure reliability

KEY TAKEAWAY Produce, publish, consume, act -- the four-stage lifecycle, plus a pull or push delivery model, defines every producer/consumer relationship. • If in doubt, start with at-least-once and make consumers idempotent -- upgrade to exactly-once only when the use case truly requires it.

AT-MOST-ONCE DELIVERY · AT-LEAST-ONCE DELIVERY · EXACTLY-ONCE PROCESSING

- REALITY AND LIMITATIONS



WHAT IT MEANS

- A message is delivered zero or one time -- never more than once
- No retries, no acknowledgments; highest performance, lowest reliability



WHEN TO USE

- Metrics, telemetry, logging, and best-effort notifications
- Never use for financial transactions or critical updates



WHY DUPLICATES HAPPEN

- Ack lost after processing, consumer timeout, or consumer failure after processing but before ack
- In every case, the broker retries delivery, and the message is delivered again



CONSUMER RESPONSIBILITY: IDEMPOTENCY

- Idempotent operations, unique message IDs with a processed table, or upsert instead of insert
- Acknowledge only after successful processing, and use retries with exponential backoff



THE REALITY: LIMITATIONS

- Network failures are unavoidable; acks can be lost after processing but before committing
- "Exactly-once" is often scoped to the messaging system, not all downstream side effects
- External side effects (emails, payments) can't be undone if duplicates occur



WHAT WORKS IN PRACTICE

- Idempotent consumers, transactional outbox/inbox, and a deduplication store
- Monitor and reconcile with metrics, alerts, and periodic reconciliation

KEY TAKEAWAY No acknowledgment, no retry: the broker sends once and moves on, which is fine for telemetry but never for a financial transaction. • If an acknowledgment is lost, delayed, or never sent, the broker retries -- so the same message may be delivered and processed more than once. • Exactly-once guarantees inside a messaging system rarely extend to external side effects like emails or payments, which can't be rolled back.

IDEMPOTENT MESSAGE CONSUMERS

Because messages may be redelivered, idempotency is not optional in a message-driven system -- it's the baseline requirement for correctness.

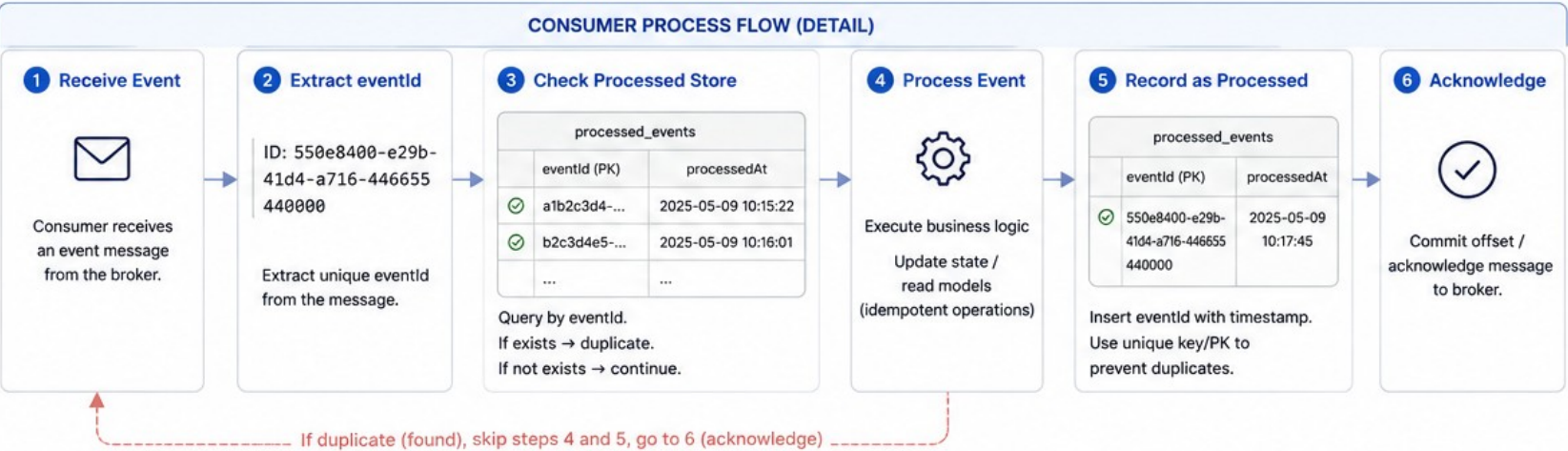
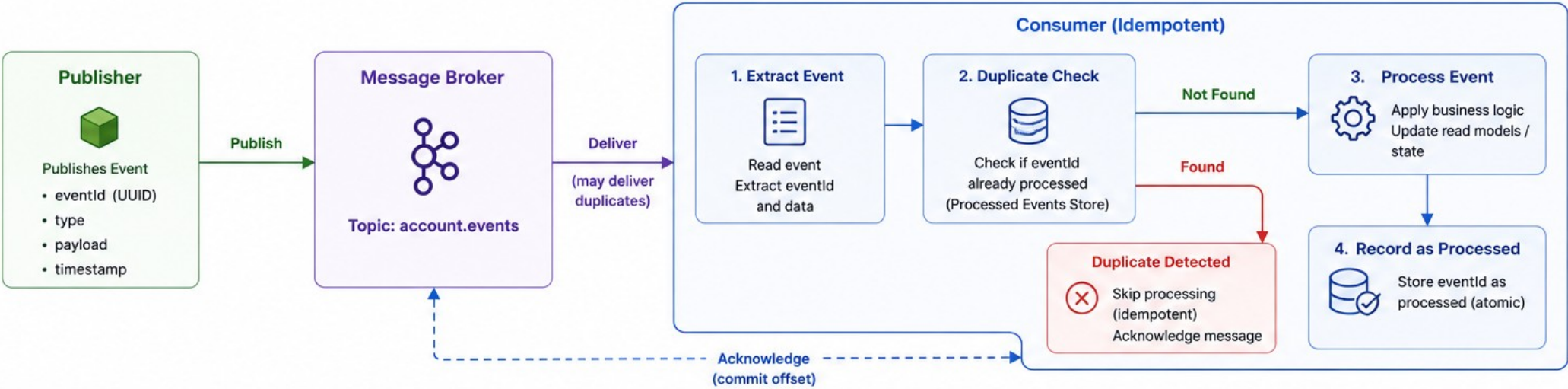


DUPLICATE HANDLING STRATEGIES

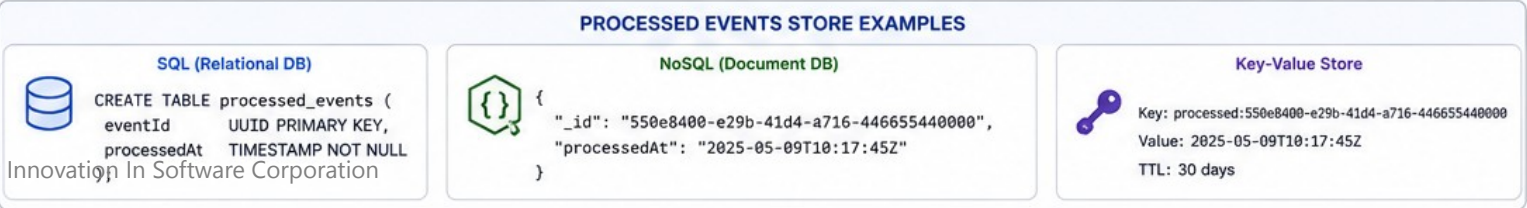
- Store processed event IDs and check before reprocessing
- Use unique business keys, upserts, or current-state checks before applying changes
- Use database constraints to enforce idempotency where appropriate

KEY TAKEAWAY Because messages may be redelivered, idempotency is not optional in a message-driven system -- it's the baseline requirement for correctness.

Idempotent Consumers and Duplicate Detection



- BEST PRACTICES**
- ✓ Use a unique eventId (UUID)
 - ✓ Store processed eventIds in a dedicated table/collection
 - ✓ Use primary key / unique constraint on eventId
 - ✓ Write business updates and processed record atomically (same transaction)
 - ✓ Make consumer logic idempotent
 - ✓ Acknowledge only after successful processing (or duplicate detection)
 - ✓ Retain processed records for a suitable retention period



MESSAGE ORDERING · MESSAGE PARTITIONING · CONSUMER GROUPS



ORDERING CONSIDERATIONS

- Kafka guarantees order within a partition, not across the whole topic
- RabbitMQ guarantees order within a single queue for a single consumer
- Design partition/routing keys deliberately when business order matters



HOW PARTITIONING WORKS

- A topic is divided into partitions, each an ordered, append-only log
- Messages with the same key always land in the same partition, preserving their order
- More partitions allow more parallel consumers, increasing overall throughput



HOW CONSUMER GROUPS WORK

- Consumers with the same group ID share a topic's partitions between them
- Each partition is assigned to exactly one consumer instance in the group at a time
- Different consumer groups each get their own full, independent copy of the event stream

KEY TAKEAWAY If a workflow depends on strict order (deposit before withdrawal), the partition key or routing key must keep those messages together. • Choosing a good partition key (like accountId) keeps all events for one entity in order while spreading load across the cluster. • Each partition is read by exactly one consumer within a group -- adding instances up to the partition count increases parallelism.

MESSAGE ACKNOWLEDGMENT AND REDELIVERY · DEAD-LETTER QUEUES · RETRY TOPICS AND BACKOFF STRATEGIES



ACKNOWLEDGMENT AND REDELIVERY FLOW

- Consumer receives a message and processes it
- On success, it sends an acknowledgment and the broker removes the message
- On failure or timeout with no ack, the broker redelivers the message



WHY DEAD-LETTER QUEUES MATTER

- Stores messages that could not be processed successfully after retries
- Prevents a single poison message from blocking the entire queue
- Allows teams to inspect, fix, and reprocess failed messages without losing them



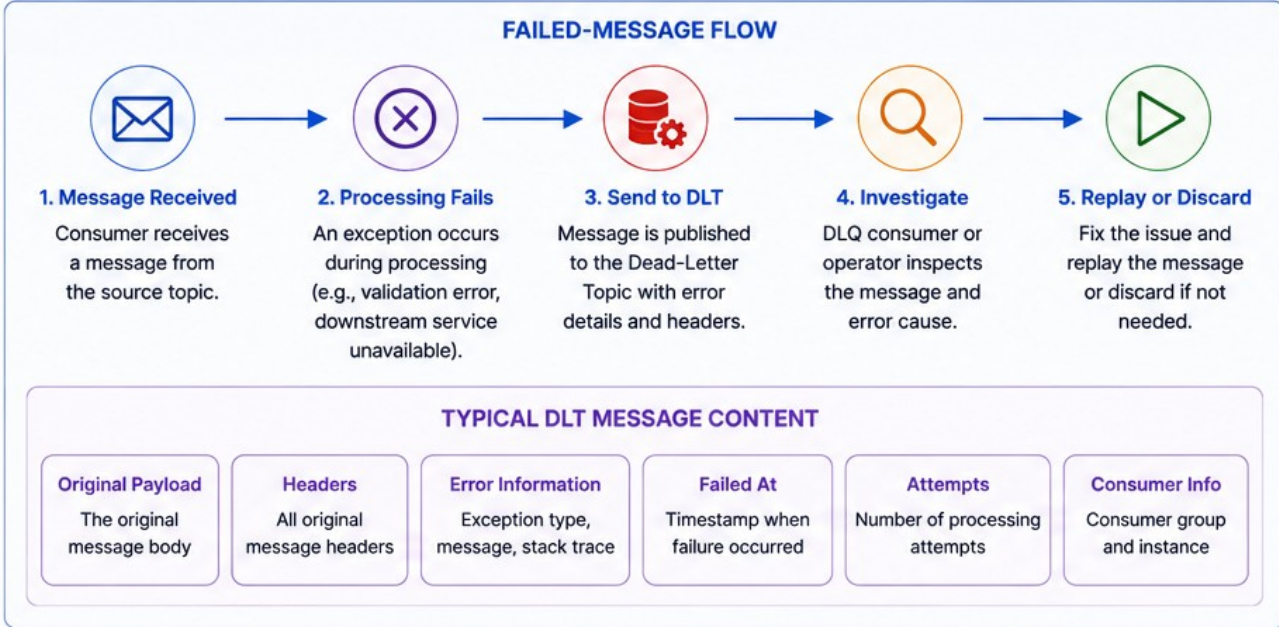
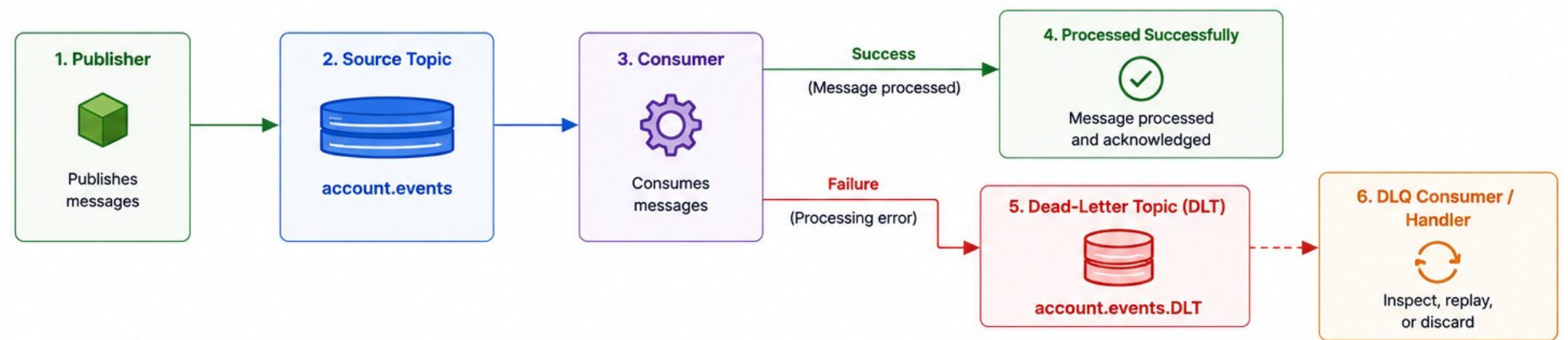
RETRY STRATEGY ELEMENTS

- **Retry topics:** separate topics/queues for staged retry attempts with increasing delay
- **Exponential backoff:** each retry waits longer than the last
- **Jitter:** randomizing delay slightly to avoid synchronized retry storms across consumers

KEY TAKEAWAY Manual acknowledgment after successful processing is safer than auto-acknowledgment, which can lose messages if the consumer crashes mid-processing.

- After a configured number of failed attempts, a message moves to the DLQ where operations teams can inspect and reprocess it manually.
- Exponential backoff with jitter spaces out retries so a temporary outage has time to recover instead of being hammered by retry storms.

Dead-Letter Topics and Failed-Message Handling



- BEST PRACTICES**
- ✓ Use DLTs to isolate and handle failed messages.
 - ✓ Include error details and context in DLT messages.
 - ✓ Set retention policy on DLTs based on business needs.
 - ✓ Monitor DLT lag and volume.
 - ✓ Build tooling for replay, inspection, and alerting.
 - ✓ Ensure idempotency when replaying messages.



CONFIGURATION EXAMPLE (CONSUMER)

<code>max.poll.records</code> 100	<code>enable.auto.commit</code> false	<code>default.dlq.topic</code> account.events.DLT	<code>max.retries</code> 3	<code>retry.backoff.ms</code> 2000
--------------------------------------	--	--	-------------------------------	---------------------------------------

POISON MESSAGES

Without a retry limit and dead-letter routing, one bad message can block an entire queue and stall every message behind it.



HANDLING POISON MESSAGES

- A message that consistently fails processing regardless of retry attempts
- Cap retry attempts, then route to a dead-letter queue instead of retrying forever
- Alert operations and validate message schemas early to prevent poison messages

KEY TAKEAWAY Without a retry limit and dead-letter routing, one bad message can block an entire queue and stall every message behind it.

 COMPATIBILITY TYPES

CHANGE TYPE	BACKWARD	FORWARD
Add optional field	Compatible	Compatible
Add field with default	Compatible	Compatible
Remove field	Not compatible	Compatible
Rename or change type	Not compatible	Not compatible

 CORE KAFKA CONCEPTS

- Producer, topic, partition, consumer, consumer group, offset, broker, cluster
- A topic is a named stream of events; partitions allow Kafka to scale and parallelize
- An offset identifies a consumer's position in a partition, enabling replay

 CORE RABBITMQ CONCEPTS

- Producer, exchange, queue, binding, routing key, consumer, dead-letter exchange
- **Exchange types:** direct, topic, fanout, and headers exchanges each route differently
- The producer sends to an exchange; the exchange routes to one or more bound queues

KEY TAKEAWAY Adding an optional field keeps old consumers working and lets new consumers read old messages -- removing or renaming a field breaks both. • Producer -> Topic -> Partitions -> Consumers, with offsets tracking each consumer group's position -- the foundation of Kafka's event-streaming model. • Producer -> Exchange -> Binding/Routing Key -> Queue -> Consumer -- exchanges give RabbitMQ its flexible routing power.

KAFKA VS. RABBITMQ · SPRING FOR APACHE KAFKA · SPRING AMQP AND RABBITMQ

✦ KAFKA VS. RABBITMQ

AREA	RABBITMQ	KAFKA
Primary model	Message broker and queues	Event streaming platform
Message storage	Queue-oriented	Durable append-only log
Replay	Limited, pattern-dependent	Built-in through offsets
Common use	Task queues, routing	Event streams, analytics, high throughput

📌 SPRING BOOT KAFKA INTEGRATION

- `@KafkaListener(topics = "transaction-events", groupId = "fraud-service")`
- `KafkaTemplate` publishes messages; `@KafkaListener` methods consume them
- Each consumer group can process the same event stream independently

📌 SPRING BOOT RABBITMQ INTEGRATION

- `RabbitTemplate` for publishing to an exchange with a routing key
- `@RabbitListener(queues = "notification.account-opened.queue")` for consuming
- Queue, exchange, and binding configuration ties the two together

KEY TAKEAWAY Choosing between them depends on whether the workload needs durable, replayable event history or flexible, routing-heavy task queues. • topics and groupId are all a listener needs to join a consumer group and start processing a topic's events. • A `RabbitTemplate.convertAndSend(exchange, routingKey, event)` call and a matching `@RabbitListener(queues=...)` method are all most services need.

TRANSACTIONAL MESSAGING CHALLENGES · THE TRANSACTIONAL OUTBOX PATTERN · CHANGE DATA CAPTURE



THE CORE PROBLEM

- A local database transaction and a broker publish cannot be committed atomically by default
- If the save succeeds but publishing fails, the event describing that save is lost



HOW THE OUTBOX PATTERN WORKS

1. Service writes the business change and an outbox row in one local transaction
2. A separate relay process reads unpublished outbox rows
3. The relay publishes each event to the broker and marks it as published



COMMON STRATEGIES

- Retry publishing, use the transactional outbox pattern, or mark events for later publishing



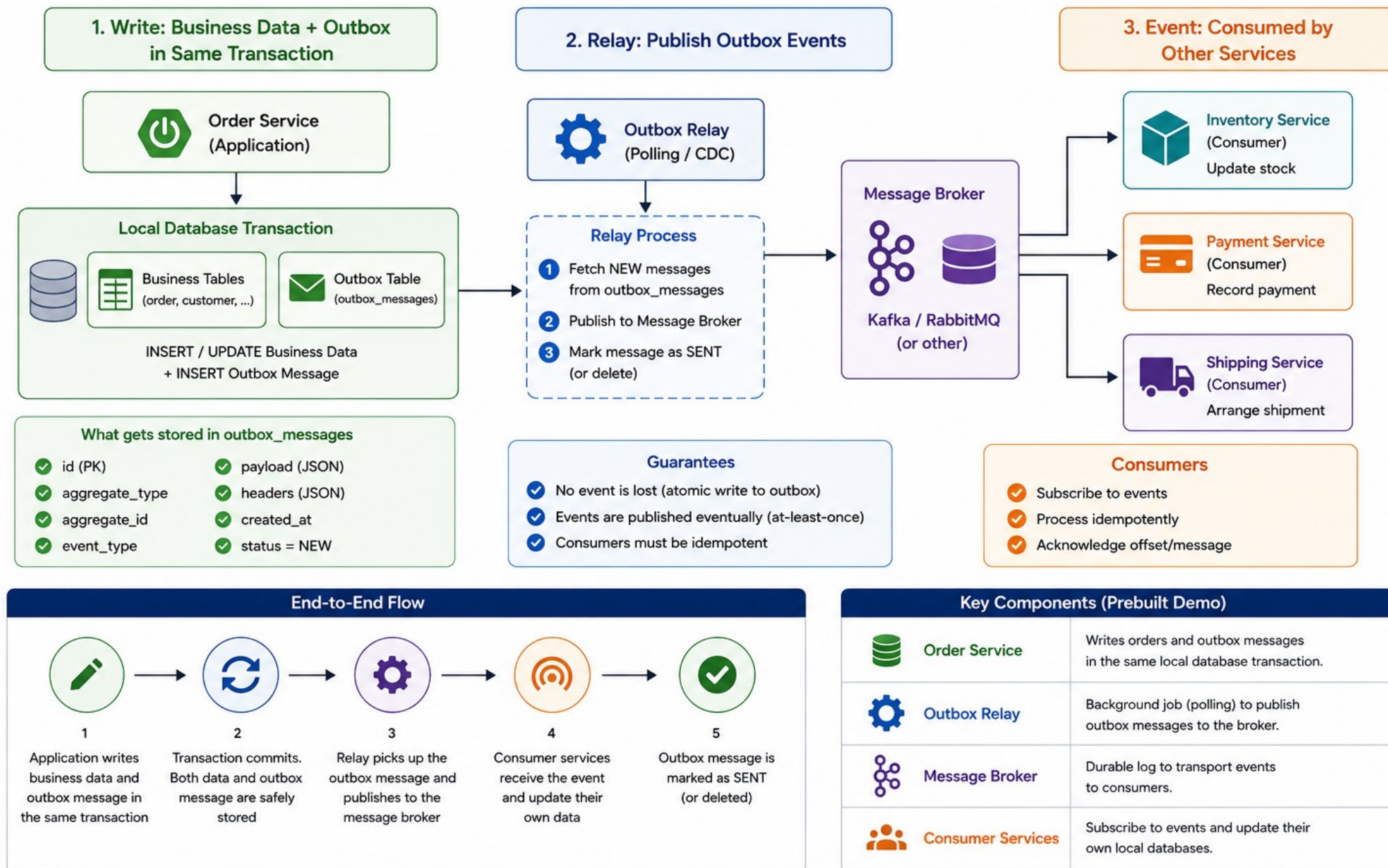
HOW CDC WORKS

- A CDC connector (e.g. Debezium) reads the database's transaction/replication log
- Each row-level change is converted into an event and published to a topic
- The application code doesn't need to explicitly publish events at all

KEY TAKEAWAY If the database commits but the broker publish fails (or vice versa), the two systems drift out of sync unless the workflow is designed for it.

- A relay process reads unpublished rows from an outbox table and publishes them to the broker, guaranteeing the event is never silently lost.
- A tool like Debezium reads the database transaction log and turns every insert, update, and delete into a Kafka event automatically.

Transactional Outbox Overview (prebuilt demo)



EVENT SOURCING OVERVIEW · EVENT SOURCING VS. EVENT-DRIVEN ARCHITECTURE



HOW EVENT SOURCING WORKS

- Every state change is persisted as an immutable event in an append-only event store
- Current state is derived by replaying (or folding) all events for an entity
- Provides a complete audit trail and enables rebuilding state at any point in time



TWO DIFFERENT CONCERNS

- **Event-driven architecture:** an integration style where services communicate via events
- **Event sourcing:** a persistence style where a service's own state is derived from an event log
- The two are complementary but solve different problems and can be adopted independently

KEY TAKEAWAY Current state is derived by replaying every event from the beginning -- giving a full, auditable history of exactly how the state was reached. • A service can be event-driven without event sourcing, and event sourcing internally without publishing every event externally -- they're independent choices.

MESSAGING OBSERVABILITY · MESSAGING SECURITY · EVENT-DRIVEN DESIGN ANTI-PATTERNS



WHAT TO MONITOR

- Correlation IDs propagated through every log, event, and consumer processing step
- **Consumer lag:** the gap between the latest available events and what's been processed
- Dead-letter queue depth, retry counts, and end-to-end message latency



SECURING MESSAGING INFRASTRUCTURE

- Require authentication and authorization for producers and consumers
- Encrypt messages in transit (TLS) and restrict network access to the broker
- Avoid including sensitive data (passwords, tokens, full account numbers) in payloads



COMMON ANTI-PATTERNS

- Publishing events before the business transaction actually succeeds
- Vague event names such as DataChanged instead of specific business facts
- Ignoring duplicate event handling and assuming strict global ordering
- Building critical workflows on messaging without any monitoring

KEY TAKEAWAY Was a message delivered? Was it processed twice? Which consumer is falling behind? Observability answers questions REST tracing alone can't. • Authentication, authorization, encryption in transit, and careful payload design keep sensitive banking events from leaking through the broker. • Vague event names, ignored duplicates, and events published before the transaction succeeds are the most common ways event-driven design goes wrong.

Integration Best Practices



1. Loose Coupling

- Communicate via well-defined contracts.
- Avoid tight dependencies between services.
- Use asynchronous events when possible.



2. Contract First

- Define API or event contracts upfront.
- Version contracts deliberately.
- Treat contracts as compatibility agreements.



3. Idempotency

- Design consumers to be idempotent.
- Use unique event IDs to detect duplicates.
- Safe to retry without side effects.



4. Reliability & Resilience

- Implement retries with exponential backoff.
- Use timeouts and circuit breakers.
- Handle failures gracefully and recover automatically.



5. Error Handling

- Handle errors explicitly and consistently.
- Use Dead-Letter Topics for failed messages.
- Capture and surface meaningful error details.



6. Data Ownership

- Each service owns its data.
- Access data via APIs or events, not direct DB access.
- Respect bounded contexts.



7. Observability

- Correlate requests/events using correlation IDs.
- Log structured, meaningful events.
- Monitor metrics and set actionable alerts.



8. Security

- Authenticate and authorize every integration.
- Encrypt data in transit (TLS) and at rest.
- Follow least privilege principles.



9. Versioning Strategy

- Version APIs and event schemas.
- Maintain backward compatibility.
- Deprecate and retire versions deliberately.



10. Performance

- Keep payloads small and efficient.
- Use pagination, filtering, and caching.
- Design for scalability and throughput.

EXERCISE 2.2 — SKETCH TRANSACTIONSUBMITTED PRODUCE/CONSUME FLOW

Module 5 checkpoint B — Producer, consumer, correlation ID.



OBJECTIVE

- Producer, consumer, correlation ID
- 10 min · design checkpoint
- **Worksheet:**
curriculum/day-02/exercises/exercise-2.2
-event-flow.md



DO THIS

- 1. After a valid REST create:** persist RECEIVED, then publish TransactionSubmitted
- 2. Envelope:** eventType, eventVersion, eventId, correlationId, occurredAt, payload
- 3.** Copy X-Correlation-Id from the HTTP request onto the event and into logs
- 4. Lab 2 implements TransactionSubmitted only. Approved / HeldForReview / Declined are later; RISK_ASSESSED is internal**



EXPECTED RESULT

- **A sequence sketch:** REST → DB → Kafka → consumer. Correlation ID on every hop.
- Lab 2 publishes and consumes TransactionSubmitted only (Compose Kafka; AMQ Streams later on OpenShift).

KEY TAKEAWAY A sequence sketch: REST → DB → Kafka → consumer. Correlation ID on every hop.

EXERCISE 2.3 — DESIGN IDEMPOTENCY AND DLQ BEHAVIOUR

Module 5 checkpoint C — Duplicate detection and failure routing.



OBJECTIVE

- Duplicate detection and failure routing
- 10 min · design checkpoint
- **Worksheet:**
curriculum/day-02/exercises/exercise-2.3-idempotency-dlq.md



DO THIS

- 1. Same eventId twice: second consume is a no-op (eventId already in processed_events)**
- 2. Poison / unreadable JSON:** retry then dead-letter — do not loop forever
- 3.** A DLT message must not invent a successful money-movement outcome
- 4.** Lab 5 Risk Assessment will be another consumer of the same topic — design for that



EXPECTED RESULT

- You can explain duplicate detection and DLT routing before you code them.
- Lab 2 uses processed_events keyed by eventId and topic transactions.submitted.DLT.

KEY TAKEAWAY You can explain duplicate detection and DLT routing before you code them.

Day 2 Summary

1. Core Concepts Covered



Data Ownership & Isolation

Database per service and why it matters.



Consistency & Communication

Eventual consistency, sagas, synchronous vs. async.



Event-Driven Integration

Domain events, brokers, Transactional Outbox pattern.



Reliability & Resilience

Idempotency, retries, error handling, and fault tolerance.

2. Hands-On Labs Completed



Account Service



Lab 1 Completed



Transaction Service



Lab 2 Completed



Both services expose REST APIs, persist data, integrate, and publish consumed events.

3. Capstone Progress



Capstone Completion

- ✓ Architecture foundation in place
- ✓ Account Service functional
- ✓ Transaction Service functional
- ✓ Event-driven integration implemented
- ✓ Ready for Risk & Notification Services

4. Key Patterns Applied



Microservices Architecture

Independent services with clear boundaries.



Repository & Data Access

Spring Data JPA for persistence.



Event-Driven Communication

Asynchronous messaging with domain events.



Resilience Patterns

Idempotency, retries, and robust error handling.

5. Tools & Technologies Used



Spring Boot 3



Spring Data JPA



Spring Validation



Kafka / RabbitMQ



OpenAPI (Swagger)



PostgreSQL



Spring Actuator



JUnit 5



Git & GitHub

6. What's Next

- Build Risk Service (Lab 3)
- Implement advanced rules & scoring
- Add Notification Service (Lab 4)
- Complete end-to-end saga flow
- Testing, hardening, and documentation



Key Takeaway

We built a solid microservices foundation with independent services, event-driven integration, and enterprise patterns—ready to extend our banking risk platform.



Journey Continues

Tomorrow: Risk Service, rules engine, and deeper saga orchestration.

Knowledge Check



1 Why do we give each microservice its own database?

- A To reduce the number of databases
- B To ensure data independence and autonomy
- C To share data more easily between services
- D To avoid writing any SQL



2 What is the main difference between synchronous and asynchronous communication?

- A Synchronous is faster and asynchronous is slower
- B Synchronous waits for a response; asynchronous does not
- C Asynchronous uses HTTP; synchronous uses TCP
- D There is no difference in reliability



3 What is the purpose of domain events?

- A To update the database schema
- B To notify other services of something that happened
- C To replace REST APIs
- D To store logs permanently



4 What problem does the Transactional Outbox pattern solve?

- A Duplicate records in a database
- B Losing events if the service crashes after commit
- C Slow database performance
- D Large message sizes



5 What is idempotency in APIs?

- A The API runs only once
- B Multiple identical requests have the same effect as one
- C The API always returns a cached response
- D The API requires authentication



6 Why do we version APIs?

- A To make the URLs look cleaner
- B To manage changes without breaking existing clients
- C To improve database performance
- D To encrypt the API requests



7 What is the main use of Actuator in Spring Boot?

- A To build REST APIs
- B To monitor and manage the application
- C To connect to the database
- D To secure endpoints with OAuth



8 What is the best practice for API error responses?

- A Return plain text messages
- B Use standardized error structure with proper HTTP status
- C Return stack traces to help the client
- D Always return 200 OK with error in body



9 What is the benefit of event-driven integration?

- A Strong consistency across services
- B Loose coupling and better scalability
- C Eliminates the need for databases
- D Faster synchronous responses



10 Which pattern helps avoid distributed transactions?

- A Two-Phase Commit
- B Saga Pattern
- C Shared Database
- D Global Locking



Self Check

Review the topics from Day 1 & Day 2.
If unsure, revisit the concept before moving forward.



Key Takeaways

Data autonomy, smart communication,
event-driven patterns, and resilience
are the foundation of our platform.

LAB 2

Lab 2 — Transaction Service

A two-service workflow using REST validation and TransactionSubmitted events.



Lab 2 Overview - Transaction Service



Purpose

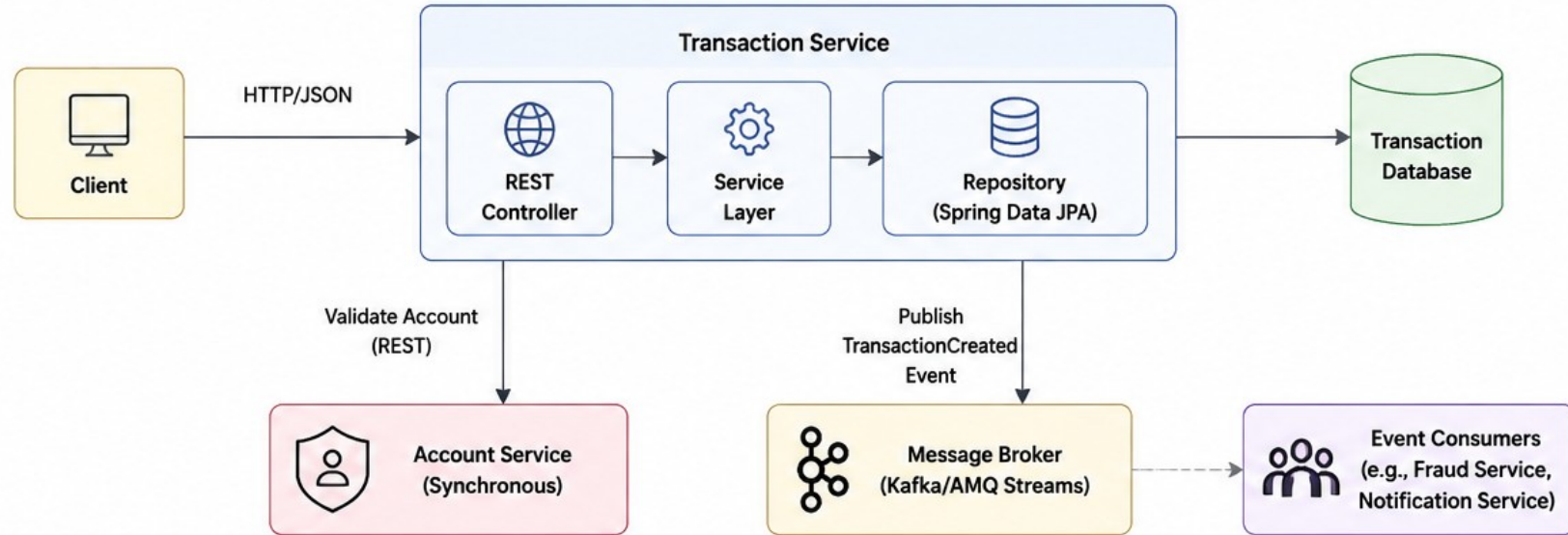
Build the Transaction Service to record and manage account transactions. Provide REST APIs and publish domain events.



Key Capabilities

- Create, read, and list transactions
- Validate account via Account Service (synchronous)
- Persist transactions (own database)
- Publish TransactionCreated event (asynchronous)
- Input validation and standardized errors
- Actuator, logging, and configuration

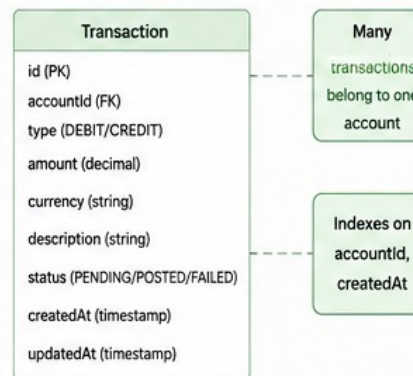
High-Level Architecture



Core API Endpoints

Method	Endpoint	Description
POST	/api/v1/transactions	Create a transaction
GET	/api/v1/transactions/{id}	Get transaction by ID
GET	/api/v1/accounts/{accountId}/transactions	List transactions by account
GET	/api/v1/transactions	List all transactions (paged)

Transaction Domain Model



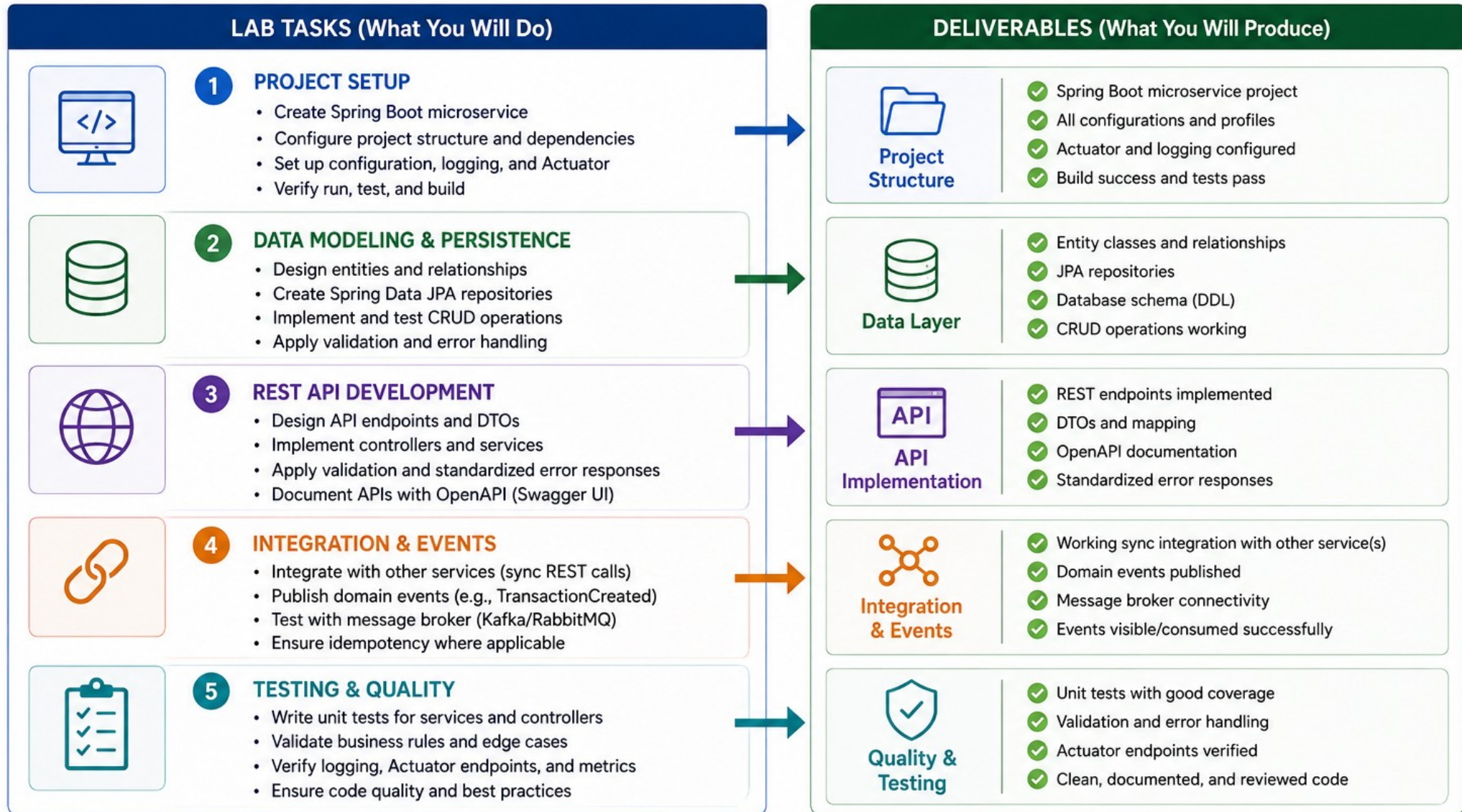
Tech Stack

- Spring Boot 3
- Spring Data JPA
- PostgreSQL (Transaction DB)
- Spring Validation
- Kafka / AMQ Streams (Events)
- Spring Actuator

Deliverables

- ✓ Transaction Service Spring Boot application
- ✓ Database schema & JPA entities
- ✓ REST APIs implemented
- ✓ Account validation (sync call)
- ✓ Publish TransactionCreated event
- ✓ Unit tests
- ✓ README with run instructions

Lab Tasks and Deliverables



END GOAL

A fully functional, tested, and documented microservice that demonstrates core enterprise patterns and is ready to integrate into the capstone platform.

LAB 2 — TRANSACTION SERVICE — OVERVIEW

Day 2 lab: what you build and how it grows the capstone.



WHAT YOU WILL FINISH WITH

- Transaction Service on port 8082
- Its own database (transaction_db on host port 5434) — no shared tables with Account Service
- REST call to Account Service before money movement is accepted
- A TransactionSubmitted event on Kafka topic transactions.submitted
- An idempotent consumer that marks the row SUBMITTED
- Poison messages on transactions.submitted.DLT
- Passing unit tests and logs that use synthetic IDs only

KEY TAKEAWAY Lab 2 builds Transaction Service -- the working code behind today's concepts, and the next layer of the banking capstone.

LAB 2 — TRANSACTION SERVICE — BEFORE YOU START

Check these before the lab clock starts.

 KNOWLEDGE YOU NEED (FROM DAY 2)

Day 2 idea	How it appears in this lab
Database per service	Account data stays in account_db. Transaction rows live only in transaction_db.
Flyway	You version the transaction schema. Hibernate does not create tables.
Synchronous REST	Before insert, GET the account. Only ACTIVE accounts may transact.
Asynchronous Kafka	After insert, publish TransactionSubmitted. The consumer updates status later.
Event envelope	eventType, eventVersion, eventId, correlationId, occurredAt, payload.
Correlation ID	Header X-Correlation-Id is copied onto the event and into logs.
Idempotent consumer	Table processed_events keyed by event_id. A replay is a no-op.
Dead-letter topic	After 2 retries, unreadable/poison messages go to transactions.submitted.DLT.
distributed transaction	You do not update Account Service's database. Eventual consistency is enough.



ENVIRONMENT BASICS

- Completed Lab 1 Account Service running on 8081 (your finished lab or labs/day-01/lab1/solution/account-service)
- Java 21
- Maven 3.9+
- Docker Desktop running (PostgreSQL and Kafka)
- Working directory: transaction-service (the folder with pom.xml)
- HTTP calls: curl.exe (not curl — PowerShell aliases curl)

KEY TAKEAWAY Settle prerequisites and environment first -- most lab time is lost to setup, not to code.

LAB 2 — TRANSACTION SERVICE — LAB STEPS

9 steps, worked in order.



LAB STEPS

- Step 1 — Run and validate the starter
- Step 2 — Apply the Flyway migration
- Step 3 — Confirm entity, repository, and schema match
- Step 4 — Validate the account with REST
- Step 5 — Persist the transaction



LAB STEPS (CONTINUED)

- Step 6 — Publish TransactionSubmitted
- Step 7 — Consume, correlate, and ignore duplicates
- Step 8 — Exercise REST, then dead-letter routing
- Step 9 — Validation, health, docs, tests, log review

KEY TAKEAWAY Work the steps in order -- each one builds on the last.

LAB 2 — TRANSACTION SERVICE — SUCCESS CRITERIA

You are done when every box below is ticked.



SUCCESS CRITERIA

- Account Service (8081) and Transaction Service (8082) both healthy
- Flyway V1 applied on transaction_db only
- POST create on an ACTIVE account returns 201 RECEIVED
- GET later shows SUBMITTED
- Non-ACTIVE account returns 409 ACCOUNT_NOT_ELIGIBLE



SUCCESS CRITERIA (CONTINUED)

- Correlation ID from the request appears in logs and on the event
- Duplicate eventId is ignored
- Poison Kafka message reaches transactions.submitted.DLT
- mvn test passes
- Logs stay synthetic and small

KEY TAKEAWAY Tick every criterion before moving on -- the next lab builds directly on this one.